> 📚 참고: 이 실습은 『파이썬 라이브러리를 활용한 머신러닝』(안드레아스 뮐러·세라 가이도 지음, 박해선 옮김, 한빛미디어) 예제와 저자의 `mglearn` 패키지(BSD 라이선스)를 바탕으로 합니다.

> 🛠️ **2026.10 수정본 (v11)** — 원본: `ch03_04_01_차원축소(1)_PCA.ipynb`
>
> - seaborn 0.12부터 `lmplot` · `regplot`에 x·y를 위치 인자로 못 넘겨서 `x='PCA1', y='PCA2'`처럼 이름을 붙였어요.
> - pandas 2.0부터 `corr()`가 글자 열(species)을 만나면 오류가 나서 `corr(numeric_only=True)`로 숫자 열만 계산해요.
> - 윈도우 폰트 경로 대신 공통 한글 폰트 설정(koreanize-matplotlib)을 씁니다.

CH03 비지도학습 - PCA

학습 내용

- 1. 왜 비지도 학습을 사용하는가?
- 2. PCA(Principal component analysis)
- 3. IRIS 데이터 셋을 이용한 PCA 예제 실습

목차

01 왜 비지도학습을 사용하는가?
02 PCA(Principal component analysis)
03 PCA를 적용한 유방암 데이터셋 시각화
04 IRIS 데이터 셋을 이용한 PCA 예제 실습

01. 왜 비지도학습을 사용하는가?

목차로 이동하기

```
가. 시각화한다.
나. 데이터를 많은 feature를 몇개의 압축적인 feature(특성)으로 줄인다.
다. 추가적인 처리(주로 지도학습에 이용하기 위해)
```

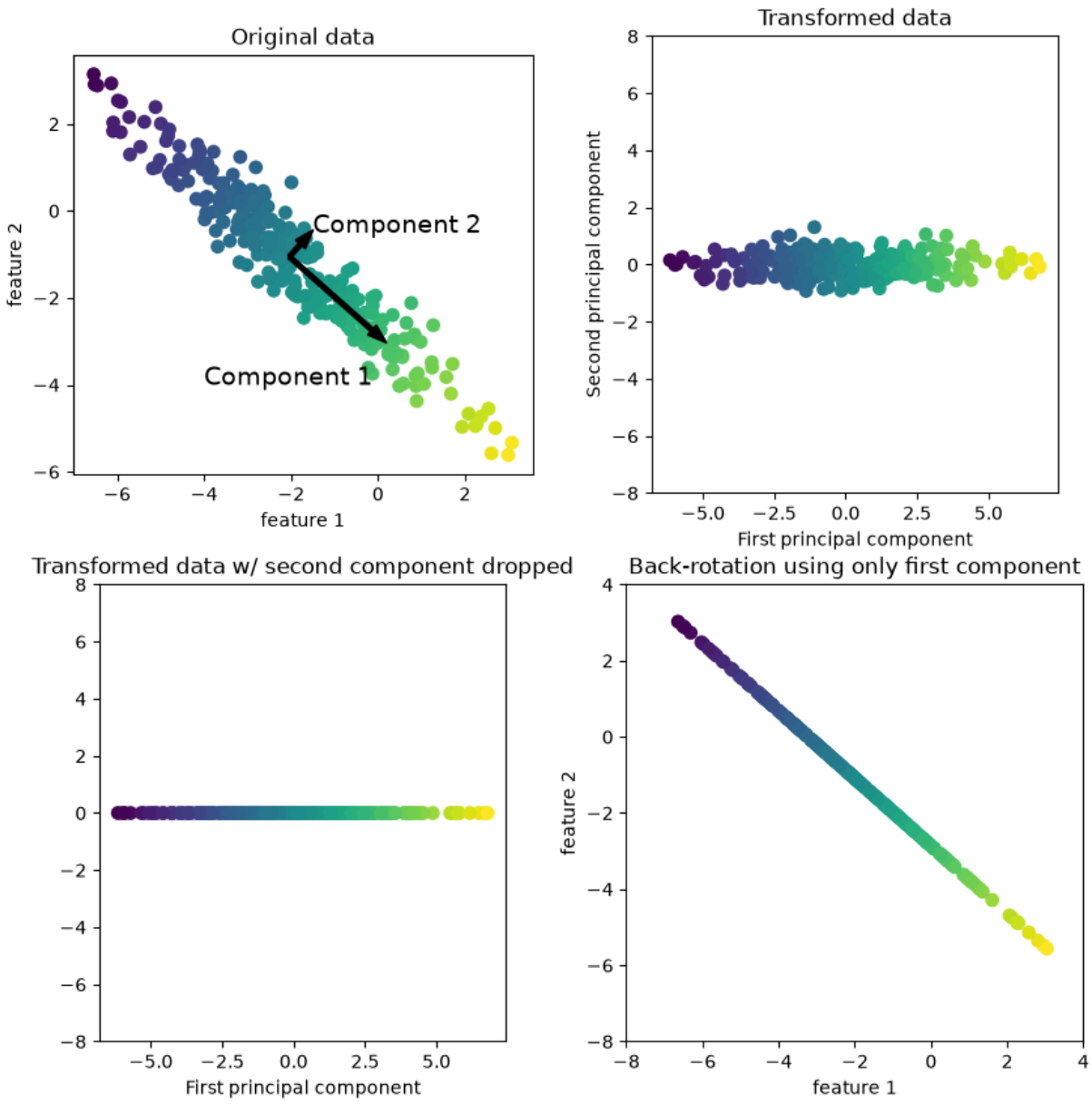
02. PCA(Principal component analysis)

목차로 이동하기

- 주성분 분석(PCA)은 특징(특성)들이 **통계적으로 상관관계가 없도록**(독립적) 데이터셋을 **회전시키는 기술**이다.
- 데이터의 회전 후에 데이터 설명도에 따라 얼마나 중요한가가 판단되고, 새로운 데이터 중의 **일부 데이터가 선택**된다.

```
In [1]: import mglearn
```

```
In [2]: mglearn.plots.plot_pca_illustration()
```



위의 네 개의 그래프는 주성분 1을 찾아가는 과정을 보여준다.

첫번째 그래프

- 원본 데이터의 주성분 1,2를 찾는다.

PCA 알고리즘

- STPE 1. PCA 알고리즘은 제일 먼저 데이터의 분산이 가장 큰 방향 **'성분1'**를 찾는다.
- 이 방향(또는 벡터)이 **데이터에서 가장 많은 정보를 담고 있는 방향이 된다.**
- 또 다른 말로 특성들의 상관관계가 가장 큰 방향입니다.
- STEP 2. 첫 번째 주성분(방향)과 직각인 방향 중, 가장 많은 정보를 담는 방향을 찾는다.(두번째 주성분)
- 만약 특성이 2개인 2차원에서는 직각방향이 하나만 존재,
- 특성이 여러개인 고차원에서는 첫번째 성분과 직각을 이루는 것은 무한히 많을 수 있다.
- STEP 3. 이와 같은 방법으로 성분을 찾아간다.

- 이런 과정을 거쳐 찾은 방향의 데이터에 있는 주된 분산의 방향이라고 해서 **주성분**이라 한다.
  - 일반적으로 원본 특성 개수만큼의 주성분이 있다.

두번째 그래프

- (가) 첫번째와 같은 데이터이지만, 주성분 1이 x축이 되도록 회전. 회전하기 전에 데이터에서 평균을 확인하여 중심을 원점을 맞춤.
- (나) PCA에 의해 회전된 두 축은 수직을 이루고, 독립적이다. 독립적이므로 상관관계 행렬(correlation matrix)이 대각선 방향을 제외하고 0이된다.
- 상관 관계 행렬의 예

```
1 0 0
0 1 0
0 0 1
```

PCA는 주성분의 일부만 남기는 차원 축소 용도로 사용된다.

세번째 그래프

- (가) 세번째 그래프는 첫 번째 주성분만 유지시킴. 2차원 -> 1차원으로 감소
  - 주성분 1(PC1) 위로 데이터를 투영시키기
  - 원본 특성중 하나만 남기는 것이 아닌 가장 유용한 방향을 찾은 주성분1만 유지.

네번째 그래프

- (가) 다시 데이터에 평균을 더해서 반대로 회전.

- (나) 이 데이터 포인트들은 원래 특징(feature) 공간에 있지만 2번째 주성분을 없앴기에 첫번째 주성분만 담고 있다.

이러한 변환은 데이터에서 **노이즈를 제거**하거나 주성분에서 유지되는 정보를 시각화

03. PCA를 적용한 유방암 데이터셋 시각화

목차로 이동하기

- 데이터 시각화 할때, 특성이 많을 때는 산점도 행렬로 시각화하여 데이터를 보는 것이 쉽지 않다.
- 이 보다 쉬운 방법은 양성과 악성 두 클래스에 대해 각 특성의 히스토그램을 그리는 것이다.

In [3]:
```
# 2026.10 수정: 윈도우 폰트 경로 대신 공통 한글 폰트 설정
import matplotlib.pyplot as plt
import matplotlib
# 한글 폰트 설정 (코랩·윈도우·맥 공통)
%pip install -q koreanize-matplotlib setuptools
import setuptools  # 2026.10 수정: Python 3.12+엔 distutils가 없어 koreanize-ma
import koreanize_matplotlib  # 나눔고딕을 matplotlib 기본 폰트로 등록
```

PCA를 적용하여 유방암 데이터 셋 시각화하기

In [4]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
import numpy as np
```

유방암 특성들을 히스토그램 시각화를 이용하여 특성을 확인(악성.양성정보 포함)

In [5]:
```
cancer = load_breast_cancer()
```

In [6]:
```
# 악성 종양(malignant) : 1
# 양성 종양(benign) : 0
cancer.target_names, cancer.target[0:15]
```

Out[6]:
```
(array(['malignant', 'benign'], dtype='<U9'),
 array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]))
```

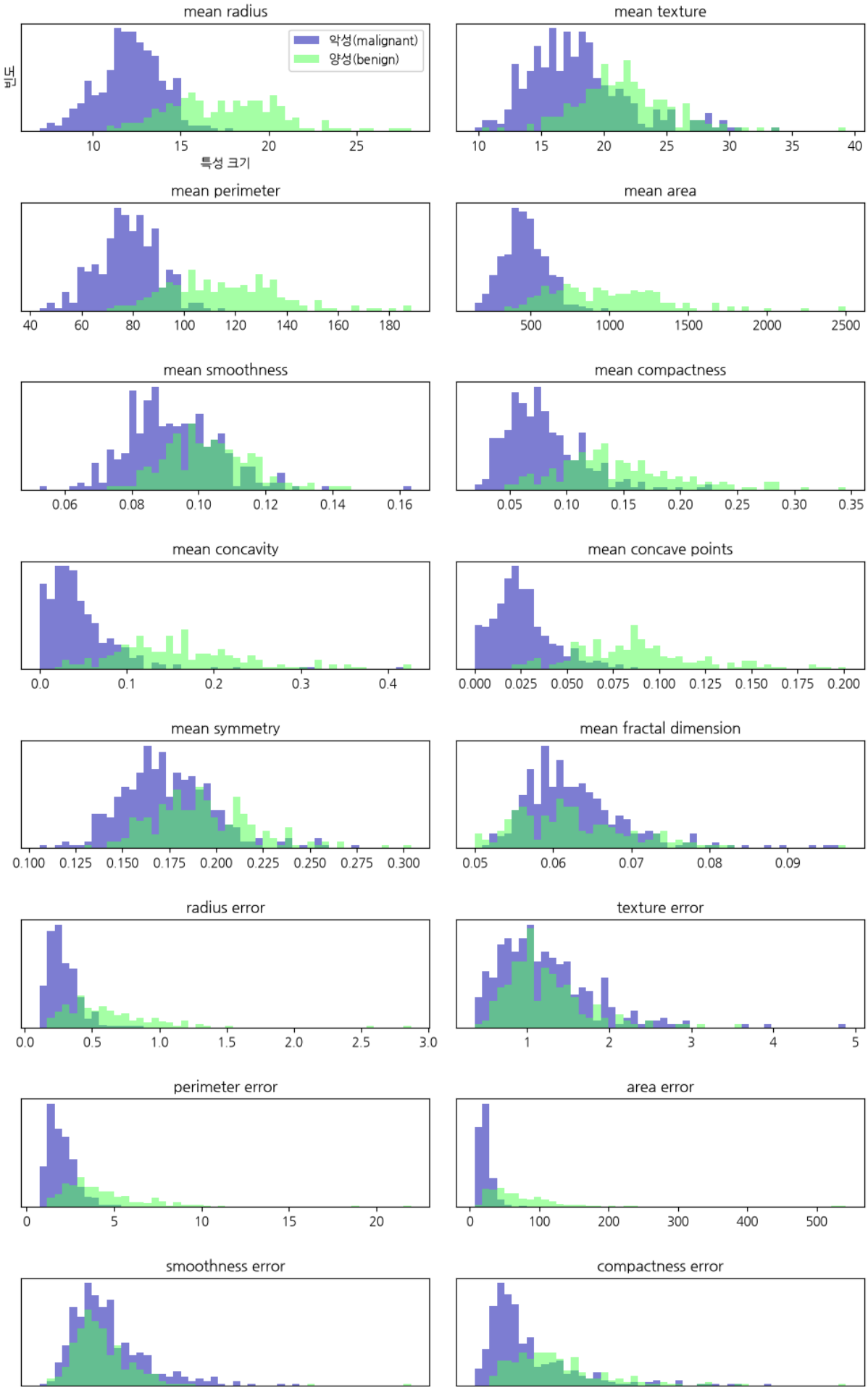
In [7]:
```
malignant = cancer.data[cancer.target == 1]
benign = cancer.data[cancer.target == 0]
```

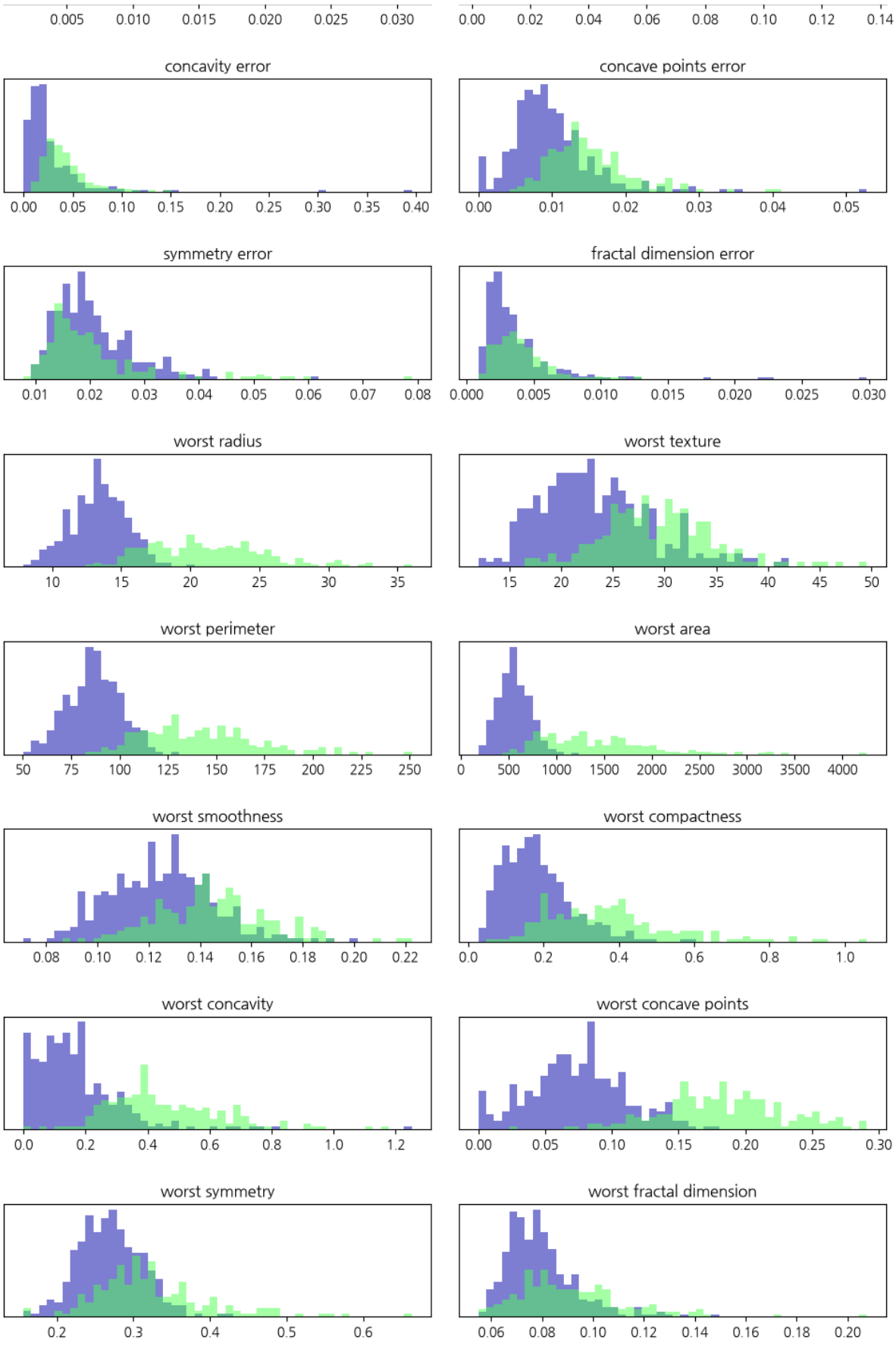
In [8]:
```
fig, axes = plt.subplots(15, 2, figsize=(10, 30))
ax = axes.ravel()

for i in range(30):
    _, bins = np.histogram(cancer.data[:, i], bins=50)
    ax[i].hist(malignant[:, i], bins=bins, color=mglearn.cm3(0), alpha=.5)
    ax[i].hist(benign[:, i], bins=bins, color=mglearn.cm3(2), alpha=.5)
    ax[i].set_title(cancer.feature_names[i])
```

```
    ax[i].set_yticks(())

ax[0].set_xlabel("특성 크기")
ax[0].set_ylabel("빈도")
ax[0].legend(["악성(malignant)", "양성(benign)"], loc="best")
fig.tight_layout()
```


mean radius
악성(malignant)
양성(benign)
빈도
10
15
20
25
특성 크기
mean texture
10
15
20
25
30
35
40
mean perimeter
40
60
80
100
120
140
160
180
mean area
500
1000
1500
2000
2500
mean smoothness
0.06
0.08
0.10
0.12
0.14
0.16
mean compactness
0.05
0.10
0.15
0.20
0.25
0.30
0.35
mean concavity
0.0
0.1
0.2
0.3
0.4
mean concave points
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
mean symmetry
0.100
0.125
0.150
0.175
0.200
0.225
0.250
0.275
0.300
mean fractal dimension
0.05
0.06
0.07
0.08
0.09
radius error
0.0
0.5
1.0
1.5
2.0
2.5
3.0
texture error
1
2
3
4
5
perimeter error
0
5
10
15
20
area error
0
100
200
300
400
500
smoothness error
compactness error



- 히스토그램을 두개 겹쳐 놓은 것으로
  - 초록색은 양성 클래스의 포인트, 푸른색은 악성 클래스의 포인트를 나타낸다.

- 이를 통해 어떤 특성이 양성과 악성을 구분하는 데 더 좋은지 가늠해 볼 수 있습니다.
  - 'smoothness error'는 악성/양성이 거의 겹쳐서 대상 특징은 별로 쓸모가 없음.
  - 'worst concave points'는 두 히스토그램이 확실히 구분되어 매우 유용한 특성.

데이터 전처리 및 PCA 변환

데이터 변환 - 표준화

In [9]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
scaler.fit(cancer.data)
X_scaled = scaler.transform(cancer.data)
```

PCA 적용

- PCA 변환을 학습하고 적용하는 것.
  - (1) PCA 객체를 생성하고, 몇개의 성분을 사용할 것인가?
  - (2) fit 메서드를 호출하여 주성분을 찾고,
  - (3) transform 메서드를 호출하여 데이터를 회전시키고 차원 축소
    - 기본값일 때는 PCA는 데이터를 회전만(이동)만 시키고 모든 주성분을 유지한다.
    - 기본적으로 100개의 특징이 있다면 주성분도 100개의 특징을 만든다.

In [10]:
```
# 데이터의 처음 두 개 주성분만 유지
# 다음과 같이 n_components 매개변수로 2개의 주성분만 사용하도록 할 수 있다.
pca = PCA(n_components=2)

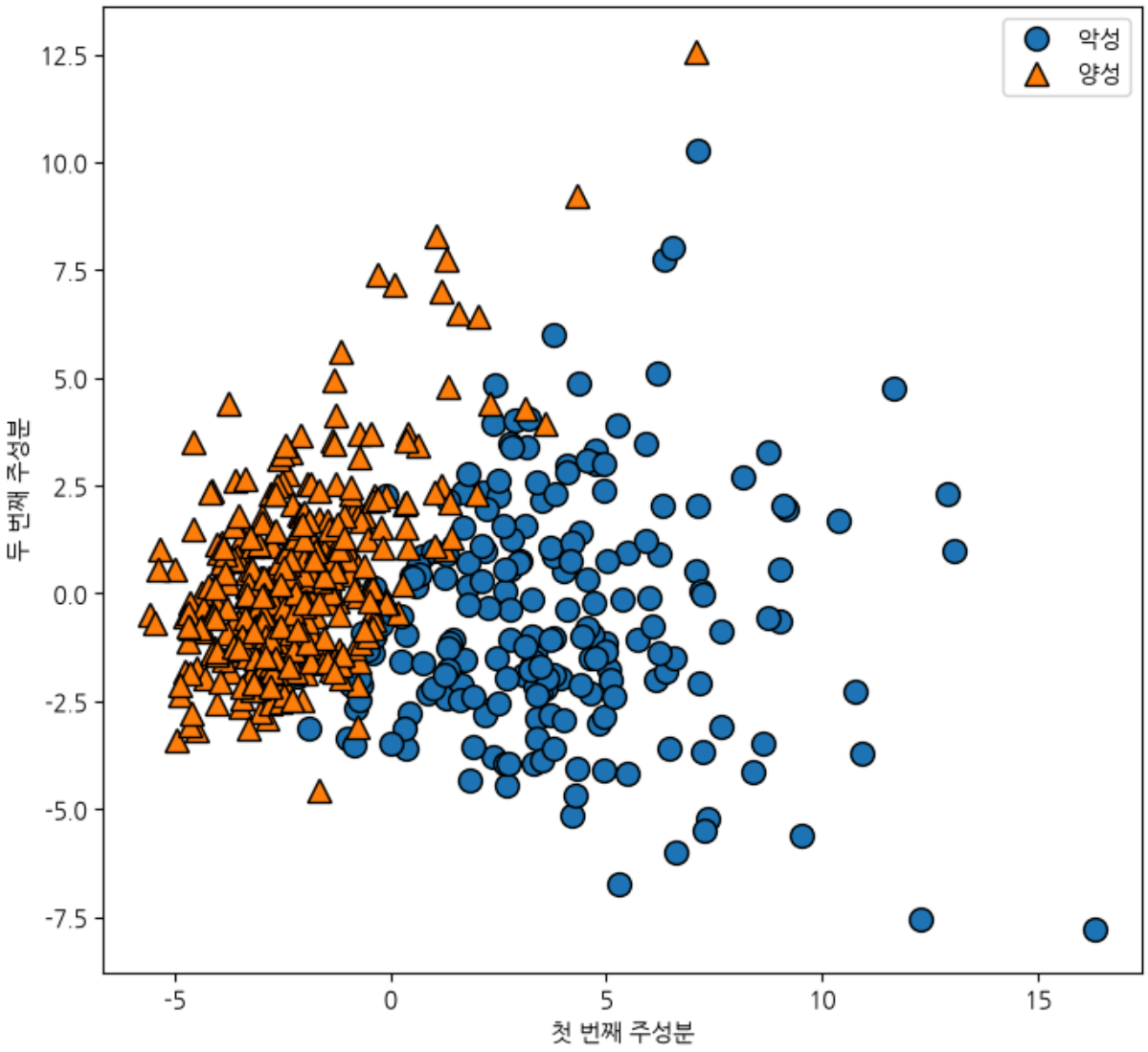
# 유방암 데이터로 PCA 모델을 생성
pca.fit(X_scaled)

# 주어진 PCA 이용하여 두 개의 주성분을 가져온다.
X_pca = pca.transform(X_scaled)
print("원본 데이터 형태: {}".format(str(X_scaled.shape)))
print("축소된 데이터 형태: {}".format(str(X_pca.shape)))
```

```
원본 데이터 형태: (569, 30)
축소된 데이터 형태: (569, 2)
```

In [11]:
```
# 클래스를 색깔로 구분하여 처음 두 개의 주성분을 그래프로 표시
plt.figure(figsize=(8, 8))
mglearn.discrete_scatter(X_pca[:, 0], X_pca[:, 1], cancer.target)
plt.legend(["악성", "양성"], loc="best")
plt.gca().set_aspect("equal")
plt.xlabel("첫 번째 주성분")
plt.ylabel("두 번째 주성분")
```

Out[11]:
```
Text(0, 0.5, '두 번째 주성분')
```



PCA로 얻어진 2개의 주성분 내용 확인

주성분

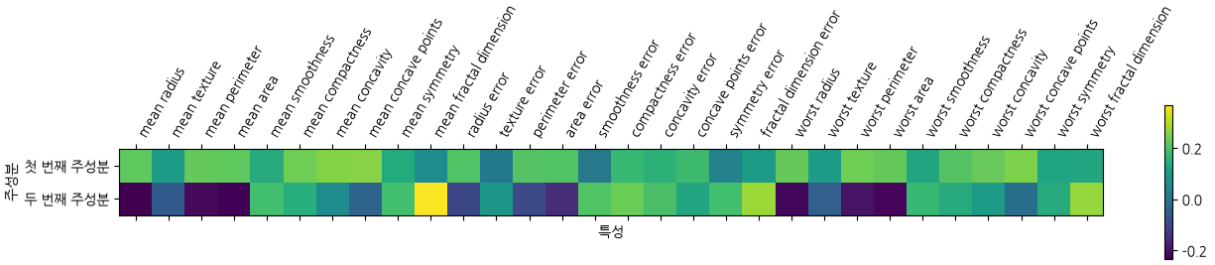
- 주성분1 : pca.components_[0]
- 주성분2 : pca.components_[1]

In [12]:
```
print("PCA 주성분 형태 ", pca.components_.shape)
print("PCA 주성분 :", pca.components_) # 두개의 주성분에 대한 결합- 회귀 계수
```

```
PCA 주성분 형태  (2, 30)
PCA 주성분 : [[ 0.21890244  0.10372458  0.22753729  0.22099499  0.14258969  0.
23928535
   0.25840048  0.26085376  0.13816696  0.06436335  0.20597878  0.01742803
   0.21132592  0.20286964  0.01453145  0.17039345  0.15358979  0.1834174
   0.04249842  0.10256832  0.22799663  0.10446933  0.23663968  0.22487053
   0.12795256  0.21009588  0.22876753  0.25088597  0.12290456  0.13178394]
 [-0.23385713 -0.05970609 -0.21518136 -0.23107671  0.18611302  0.15189161
   0.06016536 -0.0347675   0.19034877  0.36657547 -0.10555215  0.08997968
  -0.08945723 -0.15229263  0.20443045  0.2327159   0.19720728  0.13032156
   0.183848    0.28009203 -0.21986638 -0.0454673  -0.19987843 -0.21935186
   0.17230435  0.14359317  0.09796411 -0.00825724  0.14188335  0.27533947]]
```

In [13]:
```
plt.matshow(pca.components_, cmap='viridis')
plt.yticks([0, 1], ["첫 번째 주성분", "두 번째 주성분"])
plt.colorbar()
plt.xticks(range(len(cancer.feature_names)),
           cancer.feature_names, rotation=60, ha='left')
plt.xlabel("특성")
plt.ylabel("주성분")
```

Out[13]:
```
Text(0, 0.5, '주성분')
```



- 색이 진하면 진할수록 해당 주성분에 높은 영향을 끼친다.
- 하나의 주성분은 30개의 특징의 결합으로 이루어져 있다.

실습

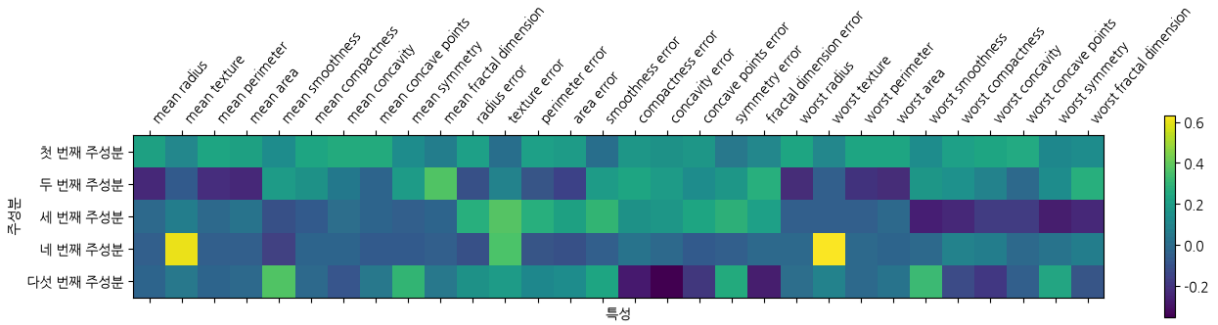
- 주성분 5개를 지정하고 이 요소에 대한 내용을 그래프로 표시해 보자.

In [14]:
```
pca = PCA(n_components=5)
pca.fit(X_scaled)
x_pca = pca.transform(X_scaled)
```

In [15]:
```
plt.matshow(pca.components_, cmap='viridis')
plt.yticks([0, 1, 2, 3, 4], ["첫 번째 주성분", "두 번째 주성분",
                             "세 번째 주성분", "네 번째 주성분",
                             "다섯 번째 주성분"])
plt.colorbar()
plt.xticks(range(len(cancer.feature_names)),
           cancer.feature_names,
           rotation=50,
           ha='left')
```

```
plt.xlabel("특성")
plt.ylabel("주성분")
```

Out[15]: Text(0, 0.5, '주성분')



04 IRIS 데이터 셋을 이용한 PCA 예제 실습

목차로 이동하기

In [16]:
```
from sklearn.decomposition import PCA
import seaborn as sns
%matplotlib inline
```

In [17]:
```
iris = sns.load_dataset("iris")

X_iris = iris.drop("species", axis=1)
y_iris = iris['species']

print(X_iris.shape, y_iris.shape)
```

```
(150, 4) (150,)
```

In [18]:
```
X_iris.head()
```

Out[18]:

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

In [19]:
```
model = PCA()
X_pca_dat = model.fit(X_iris).transform(X_iris)
print(X_pca_dat.shape, type(X_pca_dat) )
```

```
(150, 4) <class 'numpy.ndarray'>
```

In [20]:
```
X_pca_dat[0:4]
```

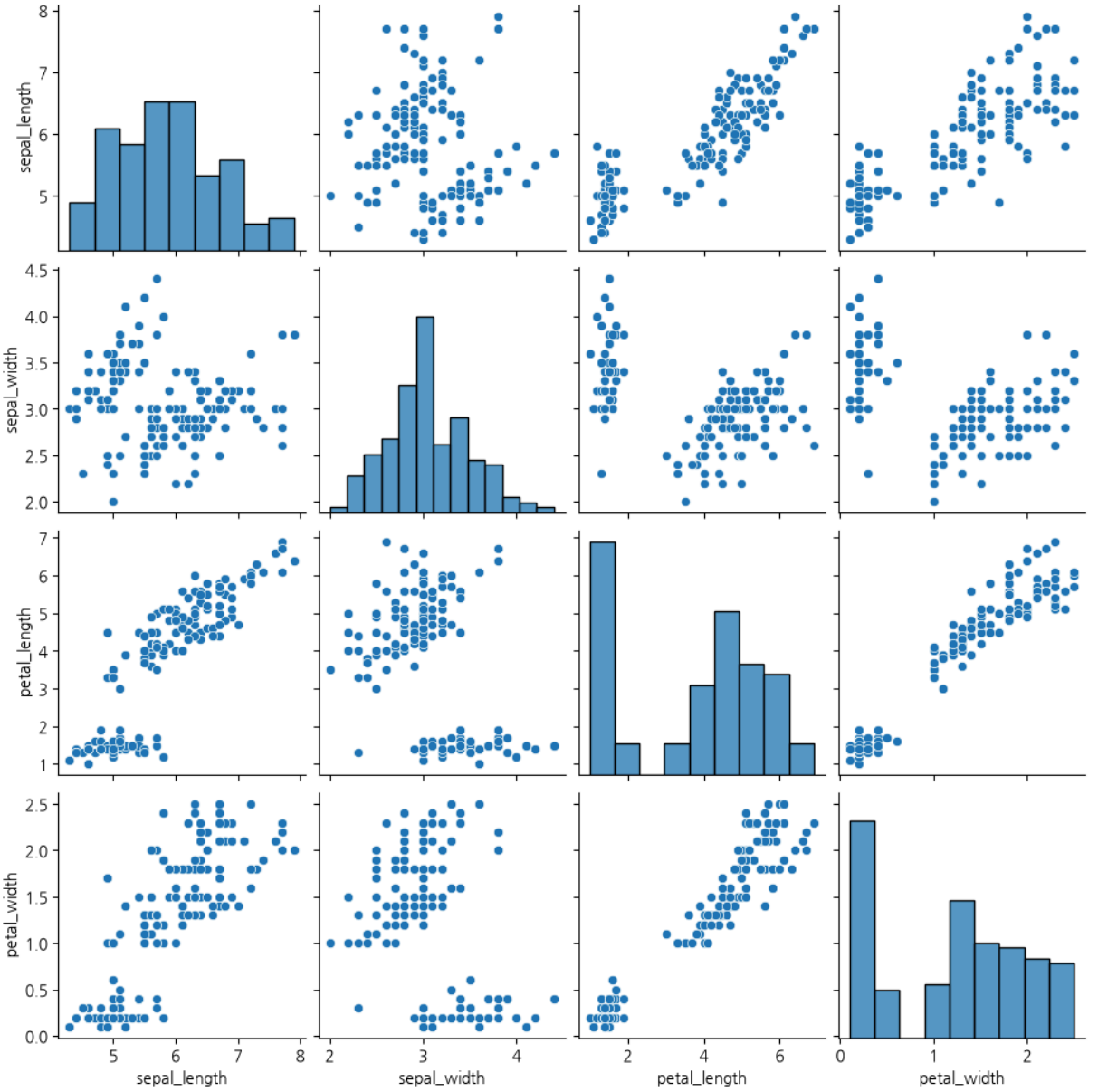
Out[20]:
```
array([[-2.68412563e+00,  3.19397247e-01, -2.79148276e-02,
         2.26243707e-03],
       [-2.71414169e+00, -1.77001225e-01, -2.10464272e-01,
         9.90265503e-02],
       [-2.88899057e+00, -1.44949426e-01,  1.79002563e-02,
         1.99683897e-02],
       [-2.74534286e+00, -3.18298979e-01,  3.15593736e-02,
        -7.55758166e-02]])
```

- n_components를 지정하지 않았을 경우, 특징의 수만큼 주성분이 만들어지게 된다.

In [21]:
```
import seaborn as sns
```

In [22]:
```
sns.pairplot(iris)
```

Out[22]:
```
<seaborn.axisgrid.PairGrid at 0x117041c70>
```

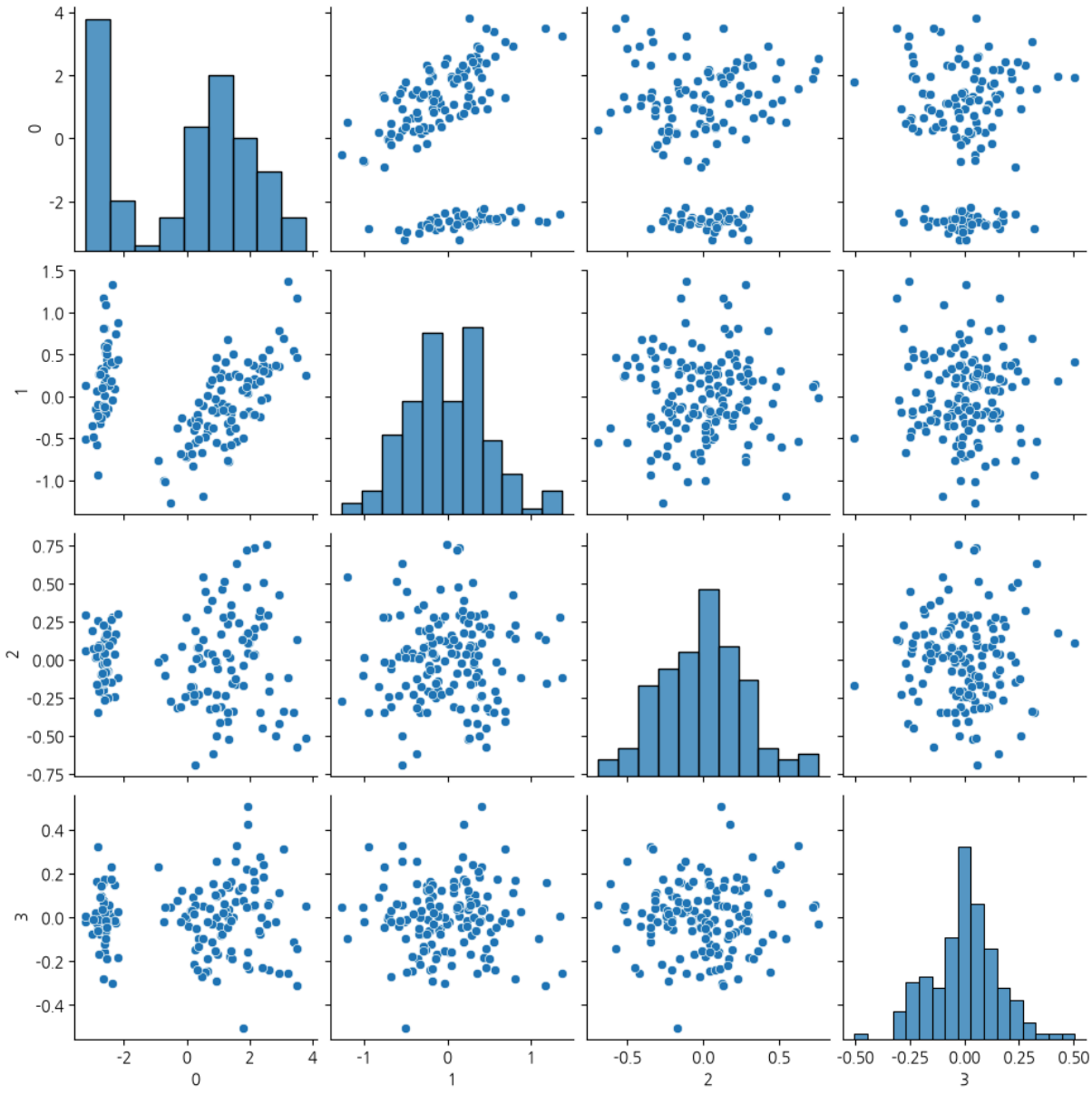


In [23]:
```
import pandas as pd
```

주성분을 데이터 프레임으로 변환 후, 시각화

In [24]:
```
df = pd.DataFrame(X_pca_dat[0:,0:])
sns.pairplot(df)
```

Out[24]:
```
<seaborn.axisgrid.PairGrid at 0x117814080>
```



- (생각해 보기) 주성분간의 관계는 어떠한가?

PCA 차원 축소로 4개의 feature를 2개의 feature로 줄여보기

- n_components 를 사용

In [25]:

```
model = PCA(n_components=2)
X_pca2 = model.fit(X_iris).transform(X_iris)
print(X_pca2.shape, type(X_pca_dat) )
print(X_pca2)
```

```
(150, 2) <class 'numpy.ndarray'>
[[-2.68412563  0.31939725]
 [-2.71414169 -0.17700123]
 [-2.88899057 -0.14494943]
 [-2.74534286 -0.31829898]
 [-2.72871654  0.32675451]
 [-2.28085963  0.74133045]
 [-2.82053775 -0.08946138]
 [-2.62614497  0.16338496]
 [-2.88638273 -0.57831175]
 [-2.6727558  -0.11377425]
 [-2.50694709  0.6450689 ]
 [-2.61275523  0.01472994]
 [-2.78610927 -0.235112  ]
 [-3.22380374 -0.51139459]
 [-2.64475039  1.17876464]
 [-2.38603903  1.33806233]
 [-2.62352788  0.81067951]
 [-2.64829671  0.31184914]
 [-2.19982032  0.87283904]
 [-2.5879864   0.51356031]
 [-2.31025622  0.39134594]
 [-2.54370523  0.43299606]
 [-3.21593942  0.13346807]
 [-2.30273318  0.09870885]
 [-2.35575405 -0.03728186]
 [-2.50666891 -0.14601688]
 [-2.46882007  0.13095149]
 [-2.56231991  0.36771886]
 [-2.63953472  0.31203998]
 [-2.63198939 -0.19696122]
 [-2.58739848 -0.20431849]
 [-2.4099325   0.41092426]
 [-2.64886233  0.81336382]
 [-2.59873675  1.09314576]
 [-2.63692688 -0.12132235]
 [-2.86624165  0.06936447]
 [-2.62523805  0.59937002]
 [-2.80068412  0.26864374]
 [-2.98050204 -0.48795834]
 [-2.59000631  0.22904384]
 [-2.77010243  0.26352753]
 [-2.84936871 -0.94096057]
 [-2.99740655 -0.34192606]
 [-2.40561449  0.18887143]
 [-2.20948924  0.43666314]
 [-2.71445143 -0.2502082 ]
 [-2.53814826  0.50377114]
 [-2.83946217 -0.22794557]
 [-2.54308575  0.57941002]
 [-2.70335978  0.10770608]
 [ 1.28482569  0.68516047]
 [ 0.93248853  0.31833364]
 [ 1.46430232  0.50426282]
 [ 0.18331772 -0.82795901]
 [ 1.08810326  0.07459068]
```

```
[ 0.64166908 -0.41824687]
[ 1.09506066  0.28346827]
[-0.74912267 -1.00489096]
[ 1.04413183  0.2283619 ]
[-0.0087454  -0.72308191]
[-0.50784088 -1.26597119]
[ 0.51169856 -0.10398124]
[ 0.26497651 -0.55003646]
[ 0.98493451 -0.12481785]
[-0.17392537 -0.25485421]
[ 0.92786078  0.46717949]
[ 0.66028376 -0.35296967]
[ 0.23610499 -0.33361077]
[ 0.94473373 -0.54314555]
[ 0.04522698 -0.58383438]
[ 1.11628318 -0.08461685]
[ 0.35788842 -0.06892503]
[ 1.29818388 -0.32778731]
[ 0.92172892 -0.18273779]
[ 0.71485333  0.14905594]
[ 0.90017437  0.32850447]
[ 1.33202444  0.24444088]
[ 1.55780216  0.26749545]
[ 0.81329065 -0.1633503 ]
[-0.30558378 -0.36826219]
[-0.06812649 -0.70517213]
[-0.18962247 -0.68028676]
[ 0.13642871 -0.31403244]
[ 1.38002644 -0.42095429]
[ 0.58800644 -0.48428742]
[ 0.80685831  0.19418231]
[ 1.22069088  0.40761959]
[ 0.81509524 -0.37203706]
[ 0.24595768 -0.2685244 ]
[ 0.16641322 -0.68192672]
[ 0.46480029 -0.67071154]
[ 0.8908152  -0.03446444]
[ 0.23054802 -0.40438585]
[-0.70453176 -1.01224823]
[ 0.35698149 -0.50491009]
[ 0.33193448 -0.21265468]
[ 0.37621565 -0.29321893]
[ 0.64257601  0.01773819]
[-0.90646986 -0.75609337]
[ 0.29900084 -0.34889781]
[ 2.53119273 -0.00984911]
[ 1.41523588 -0.57491635]
[ 2.61667602  0.34390315]
[ 1.97153105 -0.1797279 ]
[ 2.35000592 -0.04026095]
[ 3.39703874  0.55083667]
[ 0.52123224 -1.19275873]
[ 2.93258707  0.3555    ]
[ 2.32122882 -0.2438315 ]
[ 2.91675097  0.78279195]
[ 1.66177415  0.24222841]
```

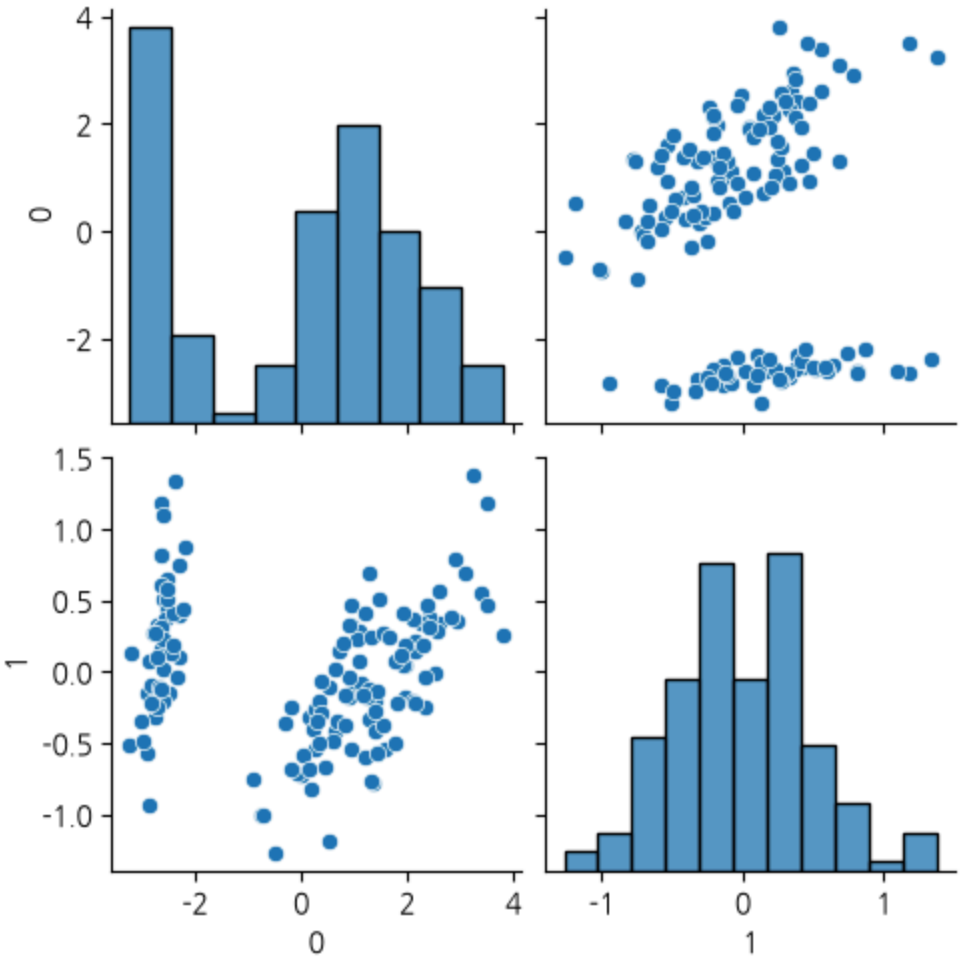
```
[ 1.80340195 -0.21563762]
[ 2.1655918   0.21627559]
[ 1.34616358 -0.77681835]
[ 1.58592822 -0.53964071]
[ 1.90445637  0.11925069]
[ 1.94968906  0.04194326]
[ 3.48705536  1.17573933]
[ 3.79564542  0.25732297]
[ 1.30079171 -0.76114964]
[ 2.42781791  0.37819601]
[ 1.19900111 -0.60609153]
[ 3.49992004  0.4606741 ]
[ 1.38876613 -0.20439933]
[ 2.2754305   0.33499061]
[ 2.61409047  0.56090136]
[ 1.25850816 -0.17970479]
[ 1.29113206 -0.11666865]
[ 2.12360872 -0.20972948]
[ 2.38800302  0.4646398 ]
[ 2.84167278  0.37526917]
[ 3.23067366  1.37416509]
[ 2.15943764 -0.21727758]
[ 1.44416124 -0.14341341]
[ 1.78129481 -0.49990168]
[ 3.07649993  0.68808568]
[ 2.14424331  0.1400642 ]
[ 1.90509815  0.04930053]
[ 1.16932634 -0.16499026]
[ 2.10761114  0.37228787]
[ 2.31415471  0.18365128]
[ 1.9222678   0.40920347]
[ 1.41523588 -0.57491635]
[ 2.56301338  0.2778626 ]
[ 2.41874618  0.3047982 ]
[ 1.94410979  0.1875323 ]
[ 1.52716661 -0.37531698]
[ 1.76434572  0.07885885]
[ 1.90094161  0.11662796]
[ 1.39018886 -0.28266094]]
```

In [26]:

```
# df = pd.DataFrame(X_pca2[0:,0:])
df = pd.DataFrame(X_pca2)
sns.pairplot(df)
df.head(10)
```

Out[26]:

	0	1
0	-2.684126	0.319397
1	-2.714142	-0.177001
2	-2.888991	-0.144949
3	-2.745343	-0.318299
4	-2.728717	0.326755
5	-2.280860	0.741330
6	-2.820538	-0.089461
7	-2.626145	0.163385
8	-2.886383	-0.578312
9	-2.672756	-0.113774



In [27]:
```
df.iloc[:,0].head()
```

Out[27]:
```
0   -2.684126
1   -2.714142
2   -2.888991
3   -2.745343
4   -2.728717
Name: 0, dtype: float64
```

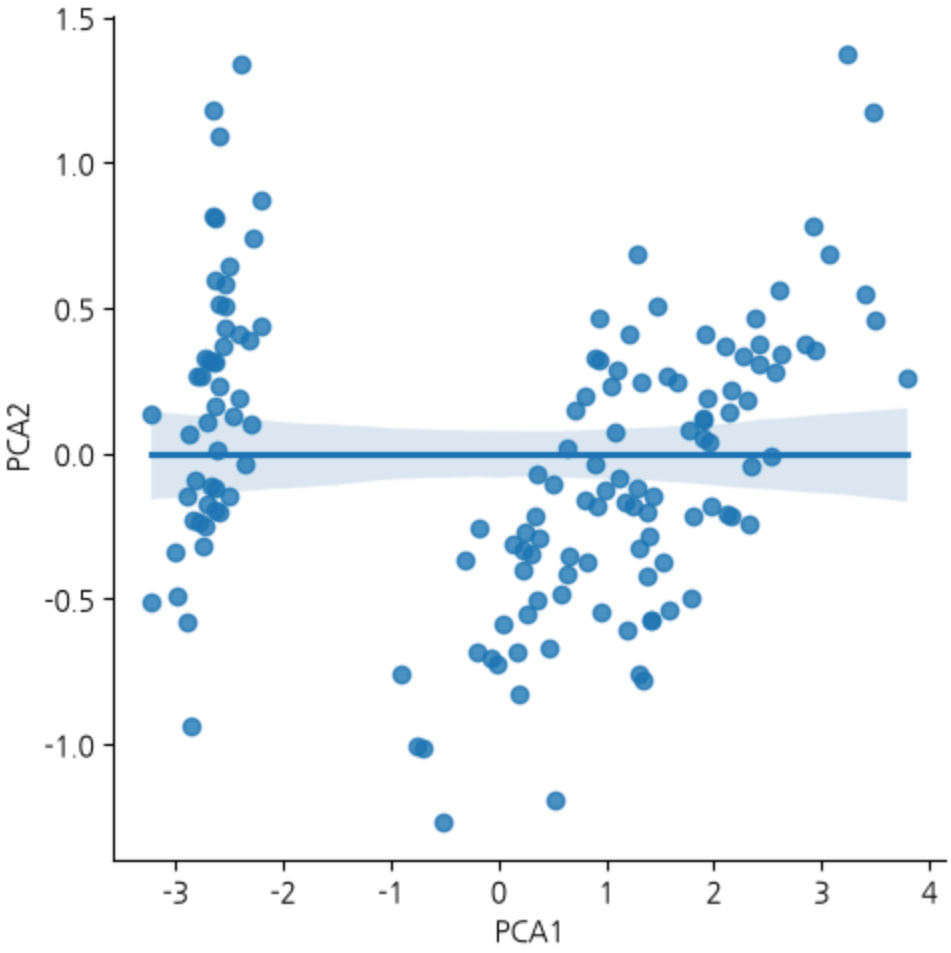
In [28]:
```
iris['PCA1'] = df.iloc[:,0]  # feature 생성
iris['PCA2'] = df.iloc[:,1]  # feature 생성
iris.head()
```

Out[28]:

	sepal_length	sepal_width	petal_length	petal_width	species	PCA1	PCA
0	5.1	3.5	1.4	0.2	setosa	-2.684126	0.31939
1	4.9	3.0	1.4	0.2	setosa	-2.714142	-0.17700
2	4.7	3.2	1.3	0.2	setosa	-2.888991	-0.14494
3	4.6	3.1	1.5	0.2	setosa	-2.745343	-0.31829
4	5.0	3.6	1.4	0.2	setosa	-2.728717	0.32675

In [29]:
```
sns.lmplot(x='PCA1', y='PCA2', data=iris, fit_reg=True)  # 2026.10 수정: x, y
```

Out[29]:
```
<seaborn.axisgrid.FacetGrid at 0x1181b0b00>
```



In [30]:
```
iris.corr(numeric_only=True)  # 2026.10 수정: pandas 2.0부터 글자 열(species)이
```

Out[30]:

	sepal_length	sepal_width	petal_length	petal_width	PCA1	
sepal_length	1.000000	-0.117570	0.871754	0.817941	8.974018e-01	3.9
sepal_width	-0.117570	1.000000	-0.428440	-0.366126	-3.987485e-01	8.2
petal_length	0.871754	-0.428440	1.000000	0.962865	9.978739e-01	-4
petal_width	0.817941	-0.366126	0.962865	1.000000	9.665475e-01	-
PCA1	0.897402	-0.398748	0.997874	0.966548	1.000000e+00	3.1
PCA2	0.390604	0.825229	-0.048381	-0.048782	3.105002e-15	1.0

위의 그래프와 상관계수를 통해 확인해 보면, PCA1과 PCA2는 상관관계를 거의 없는 독립적 관계이다.

In [31]:
```
fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(12,12))

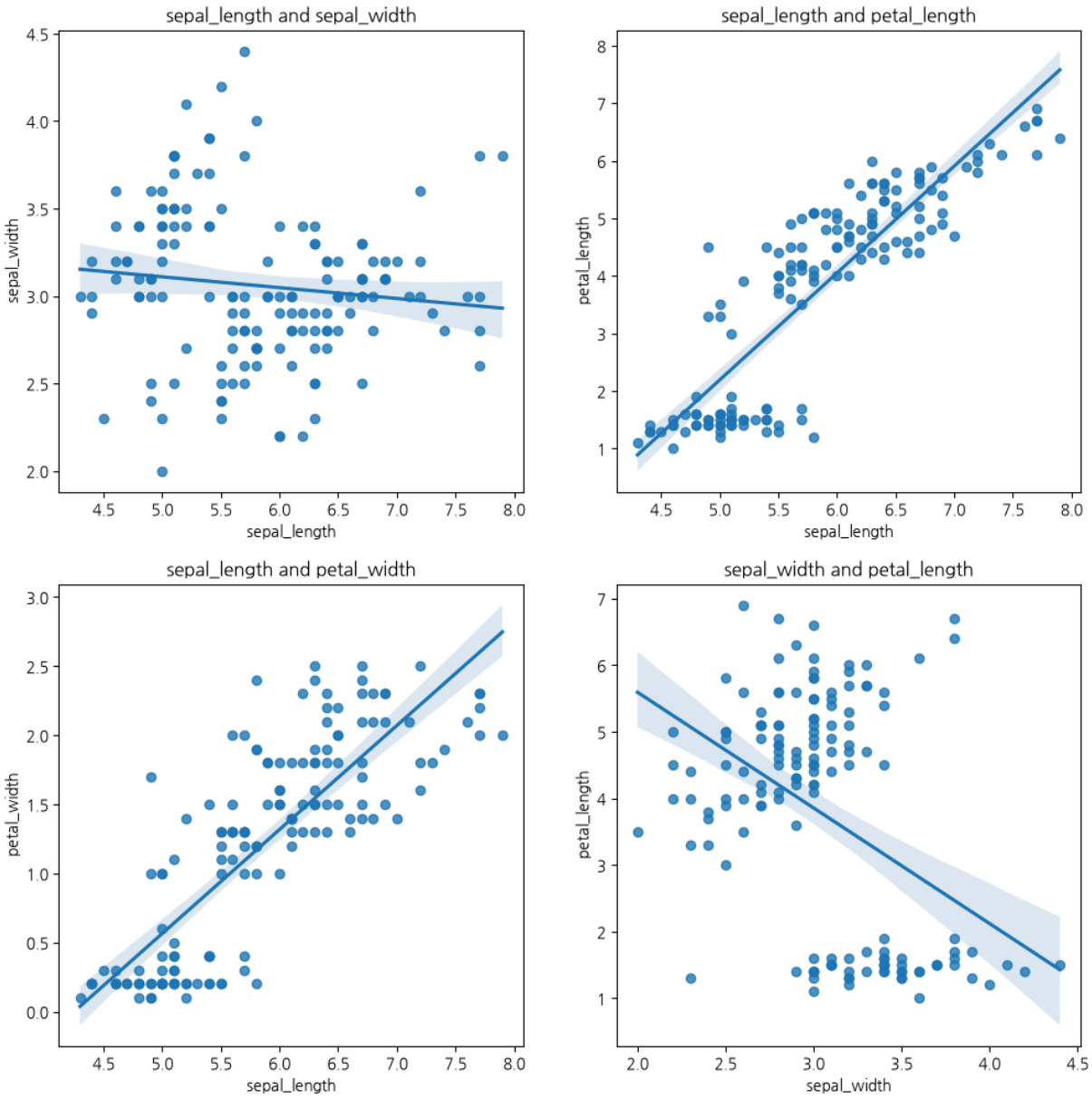
sns.regplot(x="sepal_length", y="sepal_width", data=iris, ax=ax[0][0])  # 20
ax[0][0].set_title("sepal_length and sepal_width")

sns.regplot(x="sepal_length", y="petal_length", data=iris, ax=ax[0][1])  # 2
ax[0][1].set_title("sepal_length and petal_length")

sns.regplot(x="sepal_length", y="petal_width", data=iris, ax=ax[1][0])  # 20
ax[1][0].set_title("sepal_length and petal_width")

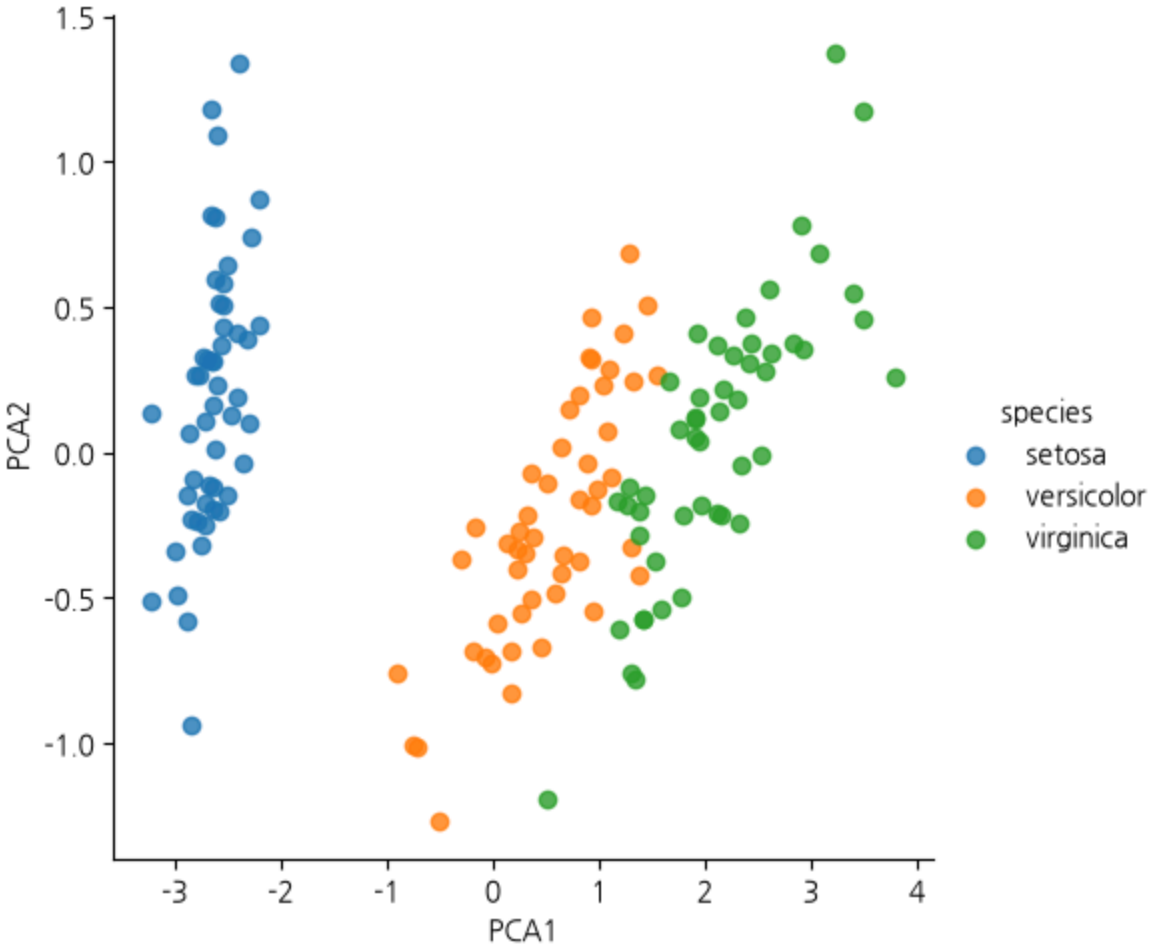
sns.regplot(x="sepal_width", y="petal_length", data=iris, ax=ax[1][1])  # 20
ax[1][1].set_title("sepal_width and petal_length")
```

Out[31]:
```
Text(0.5, 1.0, 'sepal_width and petal_length')
```



In [32]:
```
sns.lmplot(x='PCA1', y='PCA2', hue="species", data=iris, fit_reg=False)  # 2
```

Out[32]:
```
<seaborn.axisgrid.FacetGrid at 0x11819c0e0>
```



붓꽃에 대한 기본 정보가 없음에도 PCA를 활용하여 2차원으로 표현한 내용이 잘 구분되어 있음을 볼 수 있다.

REF

- Introduction to Machine Learning with Python 참조
- scikit learn PCA : https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html